

Weekly Sales Statistics Analyzer

Using VS Code · Python User Input · F-String · Type Cast · User-Defined Function · Import · Statistics Module

import module

statistics module

user input

f-string

type cast

user-defined function

1. Scenario

DataPulse Analytics — Weekly Performance Review Tool

You are a data analyst at DataPulse Analytics. Your manager needs a Python script that a regional analyst can run in the VS Code terminal. The script must:

- Import the statistics module at the top of the file.
- Prompt the analyst for their name, region, and seven daily sales figures.
- Pass the data to a user-defined function that computes all statistics inside its body.
- Return all computed values and print a formatted summary report using an f-string.

VS Code setup

1. Create a new file named `sales_stats.py`
2. Open the integrated terminal: `Ctrl+`` (Windows/Linux) or `Cmd+`` (Mac)
3. Run: `python sales_stats.py`

2. Requirements

Req 1 Import module

Add `import statistics` at the very first line of your file, before any function definitions. The statistics module is part of Python's standard library — no pip install needed.

Req 2 User input

Use `input()` to collect analyst name (string), region (string), and seven daily sales figures. Loop through 7 days, prompting the user for each value individually.

Req 3 Type casting	<code>input()</code> always returns a string. Wrap <code>float()</code> around each sales figure so the statistics functions and arithmetic work correctly. Without this, Python raises a <code>TypeError</code> .
Req 4 User-defined function	Define <code>analyze_sales(analyst, region, sales)</code> with three parameters. Compute all statistics inside the function body and return them all as a tuple.
Req 5 Statistics module	Inside the function use: <code>statistics.mean()</code> <code>statistics.median()</code> <code>statistics.mode()</code> <code>statistics.stdev()</code> . Also use <code>sum()</code> , <code>max()</code> , and <code>min()</code> — these are Python built-ins, no import needed.
Req 6 F-string output	Print the report using a triple-quoted multi-line f-string. Format all dollar amounts with <code>:.2f</code> inside the curly braces. The report must display all seven statistics plus the raw data list.

3. Hints

- Import placement**
1 `import statistics` goes on line 1, before `def`. Calling `statistics.mean()` without the import causes a `NameError` at runtime.
- Collecting seven sales figures**
2 Use a list comprehension: `sales = [float(input(f' Day {i+1}: $')) for i in range(7)]`. This runs `input()` seven times and casts each result to float in one line.
- Statistics functions**
3 `statistics.mean(sales)` · `statistics.median(sales)` · `statistics.mode(sales)` · `statistics.stdev(sales)`. All accept a list of numbers as the argument.
- Mode caution**
4 `statistics.mode()` raises `StatisticsError` if there is no single most common value. In production code, wrap it in `try/except StatisticsError` to handle multimodal data gracefully.
- Returning multiple values**
5 `return mean, median, mode, stdev, total, high, low` — Python packs these into a tuple. Unpack on the call line: `mean, median, mode, stdev, total, high, low = analyze_sales(...)`
- F-string currency formatting**
6

f"Mean: \${mean:.2f}" produces Mean: \$1296.71 — the `.2f` format spec rounds to exactly two decimal places.

4. Solution

Attempt the exercise before reading the solution. Try writing each section from memory using only the hints above.

sales_stats.py — complete program

```
import statistics                                # standard library — no install
needed

def analyze_sales(analyst, region, sales):
    mean    = statistics.mean(sales)
    median  = statistics.median(sales)
    mode    = statistics.mode(sales)
    stdev   = statistics.stdev(sales)
    total   = sum(sales)                        # built-in — no import needed
    high    = max(sales)                       # built-in
    low     = min(sales)                       # built-in
    return mean, median, mode, stdev, total, high, low

# — Collect input —————
analyst = input("Analyst name: ")
region  = input("Region: ")

print("Enter daily sales for 7 days (one per line):")
sales = [float(input(f"  Day {i+1}: $")) for i in range(7)]

# — Call the function —————
mean, median, mode, stdev, total, high, low = analyze_sales(analyst,
region, sales)

# — Print formatted report —————
print(f"""
===== Weekly Sales Statistics Report =====
Analyst : {analyst}
Region  : {region}
Data    : {sales}
-----
Total revenue : ${total:.2f}
Mean (avg)    : ${mean:.2f}
Median        : ${median:.2f}
Mode          : ${mode:.2f}
Std deviation : ${stdev:.2f}
Highest day   : ${high:.2f}
Lowest day    : ${low:.2f}
""")
```

```
=====
"""
```

Key decisions explained

`import statistics` — Placed before `def` so the module is available throughout the entire file.

`[float(input(...)) for i in range(7)]` — List comprehension collects all seven values in one line; `float()` casts each string to a number immediately.

`statistics.stdev()` — Computes sample standard deviation (divides by $n-1$). Use `statistics.pstdev()` for population standard deviation (divides by n).

`sum()` `max()` `min()` — Python built-ins; they work on any iterable and need no import.

5. Expected output

With the following input values:

Prompt	User types
Analyst name:	Maria Torres
Region:	Southeast
Day 1: \$	1200.00
Day 2: \$	1450.50
Day 3: \$	980.00
Day 4: \$	1450.50
Day 5: \$	1320.75
Day 6: \$	1100.00
Day 7: \$	1575.25

The terminal prints:

```
===== Weekly Sales Statistics Report =====
Analyst : Maria Torres
Region  : Southeast
Data    : [1200.0, 1450.5, 980.0, 1450.5, 1320.75, 1100.0, 1575.25]
-----
Total revenue : $9077.00
Mean (avg)    : $1296.71
Median       : $1320.75
```

```
Mode          : $1450.50
Std deviation : $209.34
Highest day   : $1575.25
Lowest day    : $980.00
=====
```

Statistics summary

Statistic	Value	Python function
Total revenue	\$9,077.00	<code>sum(sales)</code>
Mean (average)	\$1,296.71	<code>statistics.mean(sales)</code>
Median	\$1,320.75	<code>statistics.median(sales)</code>
Mode	\$1,450.50	<code>statistics.mode(sales)</code>
Std deviation	\$209.34	<code>statistics.stdev(sales)</code>
Highest day	\$1,575.25	<code>max(sales)</code>
Lowest day	\$980.00	<code>min(sales)</code>
Range (hi - lo)	\$595.25	<code>max(sales) - min(sales)</code>

6. Statistics module quick reference

The table below summarises every function used in this exercise and when to choose each one.

Function	Returns	Notes
<code>statistics.mean(data)</code>	float	Arithmetic average. Equivalent to <code>sum(data)/len(data)</code> .
<code>statistics.median(data)</code>	float	Middle value when sorted. Not affected by outliers.
<code>statistics.mode(data)</code>	value	Most common value. Raises <code>StatisticsError</code> if no unique mode.
<code>statistics.stdev(data)</code>	float	Sample std deviation (n-1). Use when data is a sample.
<code>statistics.pstdev(data)</code>	float	Population std deviation (n). Use when data is the full population.
<code>statistics.variance(data)</code>	float	Sample variance (square of stdev).

7. Extension challenges

Once your base program runs correctly, try these extensions:

4. Add `statistics.variance()` to the report and explain in a comment how variance relates to standard deviation.
5. Wrap `statistics.mode()` in a `try/except StatisticsError` block and print a message if no unique mode exists.
6. Calculate the range: `high - low`, and add it to the report as an eighth statistic.
7. Use a `for` loop to run the program for multiple regions and compare their means at the end.
8. Write the report to a text file using `open('report.txt', 'w')` instead of (or in addition to) printing.